

Rust <-> C++ FFI Seam — Remaining Work

Andrew Jensen

2026-08-25

1 Legend

Four parallel scout passes produced this list (bridge surface, C++ call path, contract-vs-reality, idiom review). Every item carries [line](#) evidence so you can jump straight there. Priorities:

[#A] Blocks **any** call crossing the boundary (the seam is inert until these land)

[#B] Blocks numeric parity / Tier-3 replay (the port runs but won't match C++)

[#C] Wiring, hygiene, and idiomatic polish

Current state in one line: the bridge compiles and is linked everywhere, but exports zero functions, is included by zero C++ files, and has two latent divergence bugs that would fail a parity gate even if it ran.

2 TODO Make the boundary actually crossable boundary

2.1 TODO Declare `build_cpp_cost` in the bridge [BLOCKER]

Rust can never obtain a `UniquePtr<CppCost>` today — it exists only as a C++ free fn ([cost.h:54](#)), absent from the `unsafe extern "C++" {` block ([lib.rs:26-33](#)). Note `cxx` cannot take `std::function` directly: either pass plain scalars to a C++ factory, or have C++ construct the `CppCost` and hand it across (injection pattern). Decide which direction owns construction before writing this line.

2.1.1 Walkthrough — getting a `CppCost` into Rust's hands

Where you're starting: the bridge knows `CppCost` EXISTS but has no way to CREATE one:

```
1 // rust/direct-rs/src/lib.rs (current)
2 unsafe extern "C++" {
3     include!("domain/cost.h");
4     include!("domain/data_structures_6D.h");
5     type CppCost;
6     type Point6D;
7     pub fn evaluate(self: &CppCost, point: &Point6D) -> f64;
8 }
```

and C++ offers exactly one factory, whose parameter `cxx` cannot spell:

```
1 // include/domain/cost.h:54
2 std::unique_ptr<CppCost> build_cpp_cost(CppCost::CostFunction fn);
3 //                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ std::function<double(const Point6D&>
4 //                               cxx has NO representation for std::function - period.
```

Q1 — warm-up. Why can't the declaration above be fixed by simply adding `fn build_cpp_cost(fn: ???) -> UniquePtr<CppCost>;`? Name the exact reason `std::function` is unspellable, and therefore what property every argument of EVERY function you ever declare in an extern "C++" block must have.

Q2 — who owns the real cost at runtime? Open `optimizer_manager.cpp:1745-1788` (`BuildGpuCostAdapter`) and identify what the lambda captures (`gpu_principal_model__`, `calibration__`, `stage_manager`). Question: could ANY of those captures cross the FFI as arguments to a Rust-called factory? If not, what does that force the construction direction to be — C++ builds and hands over, or Rust calls in with plain args?

Q3 — the two doors. `cxx` gives you exactly two ways for a `UniquePtr<CppCost>` to come into existence on the Rust side. Write both declarative forms from memory, then check yourself:

```
1 // Door 1: free factory wrapping a (possibly arg-less) C++ fn
2 ___ build_cpp_cost(___ ) -> UniquePtr<CppCost>;
3
4 // Door 2: declare the C++ CONSTRUCTOR itself
5 // (cxx maps `fn new(...) -> UniquePtr<CppCost>` to CppCost(...))
6 ___ new(___ ) -> UniquePtr<CppCost>;
```

Hint: `cost.h:38` shows `CppCost` has TWO ctors — default and `explicit(CostFunction)`. Only one of them is declarable. Why?

Q4 — the argument problem. Whichever door you pick, the factory needs to know WHICH cost to seal. Brainstorm three candidate argument shapes (nothing / a small enum tag / something richer), and for each ask: “could the GPU adapter’s captured state be recovered from this at the C++ end?” Record which one survives.

Q5 — write it. Add your chosen declaration to `lib.rs` (don’t forget use `cxx::UniquePtr`), plus whatever minimal arg-less overload `cost.h` needs. Compile with cargo check.

Decision to record: construction direction (who calls whom), and whether `build_cppcost` stays in `cost.h` or gets a sibling. One sentence under this heading when you’ve decided.

Checkpoint: a `#[cfg(test)]` test in the crate can obtain a `UniquePtr<CppCost>` without panicking (it may be unbound — that’s fine, the `IsBound` guard item covers that separately).

2.2 TODO Give Rust a way to make a Point6D [BLOCKER]

Both `Pose` and `Point6D` are declared opaque (`lib.rs:29-30`) and no conversion exists anywhere on either side. Even with a bound `CppCost`, Rust has no instance to hand `evaluate(&Point6D)`. Fields match 1:1 (`x, y, z, xa, ya, za`, `direct_datastorage.rs:28-35` vs `data_structures6D.h:25-40`). Recommended shape: declare the six-double ctor in the extern block and write one manual converter `pose_to_point(&Pose) -> UniquePtr<Point6D>` — `impl` From is impossible (orphan rule + `cxx` semantics), and a shared third coordinate type just to avoid six field copies isn’t worth it.

2.2.1 Walkthrough — Pose -> Point6D, the honest way

Where you're starting: both sides describe the same six doubles, and neither side can see the other's type:

Pose (rust/direct-rs/src/direct_datastorage.rs:28-35): x, y, z, xa, ya, za : f64 Point6D (include/domain/data_structures6D.h:25-40): x, y, z, xa, ya, za : double (+ ctor taking all six)

Q1 — kill the tempting shortcut. Two reasons 'impl From<Pose> for Point6D' cannot exist. Reason one is a general Rust rule about traits and types from different crates — state it (hint: coherence/orphan). Reason two is specific to cxx: foreign types come into existence only through `__`. Fill the blank.

Q2 — evaluate the shared-struct escape hatch. A 'shared struct Pose6D' in the bridge would let values flow both directions without opaque machinery. List what it costs (third coordinate type forever; every crossing still copies six doubles; Pose stays the internal truth) and decide if any benefit remains. (Scouts' verdict: no — but convince yourself.)

Q3 — declare the constructor. cxx lets an extern "C++" block map the C++ constructor by convention. Fill in the blanks:

```
1  unsafe extern "C++" {
2      type Point6D;
3      pub fn ___(x: f64, y: f64, z: f64, xa: f64, ya: f64, za: f64)
4          -> UniquePtr<Point6D>;
5  }
```

(If unsure of the magic name, the cxx book's opaque-types chapter is one page; the handshake doc's Appendix H also points at 'cargo expand'.)

Q4 — write the one-way converter. Where it lives matters: the crate already contains a dead 'use crate::ffi::CppType;' marker at direct_datastorage.rs:5 suggesting glue was PLANNED to live there. Fill in:

```
1  pub(crate) fn pose_to_point(p: &Pose) -> UniquePtr<Point6D> {
2      Point6D::new(p.____, p.____, p.____, p.____, p.____, p.____)
3  }
```

Then decide: delete that dead import, or is this fn its intended use?

Q5 — think ahead to the return trip. When run() finishes, the BEST pose must travel back to C++, which speaks Point6D. Trace the flow and answer: does Rust ever need point_{to_pose}? Or does the best-pose exit via a different vehicle entirely (see the Export-surface walkthrough's RunOutcome struct)?

Checkpoint: cargo check green; then pixi run build so the C++ half of the regenerated bridge compiles too (corrosion regenerates lib.rs.h — cargo check alone never exercises that half).

2.3 TODO Export the DirectOptimizer surface (zero functions cross today)

The extern "Rust" block declares bare types only (lib.rs:21-24). The real API waiting to cross: new / new_with_strat (direct_optimizer.rs:36,49), run (:82), best (:128). Two shape problems en route:

- run<T: Cost> is generic — generics don't cross cxx. You need one concrete adapter type wrapping &CppCost that implements the batch Cost trait (&[Pose] -> Vec<f64>, cost.rs:3-5) by looping evaluate per pose. That adapter is the D4 gap.
- run~/~best return (Pose, f64) tuples — cxx supports neither tuples nor an unbridged Pose. Introduce BestPoint { pose, value } internally first (one refactor, see idiom section), then expose it as a shared struct or two accessor methods.

2.3.1 Walkthrough — opening the extern “Rust” door, one shim at a time

Where you're starting: the API exists and works (pure-Rust tests prove it); the bridge just repeats the type names:

```

1 extern "Rust" {
2     type DirectOptimizer;      // <- everything below must be ADDED
3     type POHSettings;
4 }
5 // waiting behind the wall (direct_optimizer.rs):
6 // new(range, starting_point, budget)           (:36)
7 // new_with_strat(..., poh_strat)               (:49)
8 // run<T: Cost>(&mut self, cost: T) -> (Pose, f64) (:82)
9 // best(&self) -> (Pose, f64)                   (:128)

```

Q1 — constructors. ‘new’ takes Pose BY VALUE — unbridged. You have three options: (a) take six f64s and assemble Pose inside a shim fn; (b) take &Point6D (yes — extern “C++” types may appear in extern “Rust” signatures) and convert inside; (c) something with BestPoint. For each, ask “how many conversions happen per call?” Pick one for the constructor shim and write its full signature as it will appear in the extern “Rust” block.

Q2 — the generic wall. run<T: Cost> monomorphizes per T; cxx emits exactly one symbol per declared fn. So the bridge can never say ‘run’. Design the single concrete adapter. Skeleton with three blanks:

```

1 struct CppCostBridge<'a> { cost: &'a CppCost }
2
3 impl Cost for CppCostBridge<'_> {
4     fn eval(&self, poses: &[Pose]) -> Vec<f64> {
5         poses.iter().map(|p| {
6             let pt = pose_to_point(p);
7             --- // <- the ONE cxx call that makes this whole thing work
8         }).collect()
9     }
10 }
11
12 pub fn direct_rs_run(this: Pin<&mut DirectOptimizer>, cost: &CppCost) -> --- {
13     --- // <- connect adapter + inherent method
14 }

```

Sub-questions: (i) why must eval convert rather than the loop outside? (ii) &mut self across cxx — does the exported form want ‘self: &mut DirectOptimizer’ method syntax (C++ calls `opt.run(...)`) or a free fn? Which does the C++ engineer prefer reading? (iii) what does cxx demand instead of &mut for non-Send opaque types — check `Pin<&mut>`.

Q3 — the tuple wall. Before anything returns, land the idiom-section refactor: `BestPoint { pose: Pose, value: f64 }`. Decide NOW whether internal run/best switch to it (cleaner, touches properties tests) or the shim wraps tuples at the edge (zero internal churn). Both defensible — pick and justify in one sentence.

Q4 — the return vehicle. C++ ultimately wants six doubles + score + calls used. Draft the shared struct (shared structs live OUTSIDE the extern blocks):

```
1 struct RunOutcome {
2     __: f64, /* five more axes */
3     value: f64,
4     calls: u32,    // why does the cumulative-budget ladder NEED this?
5 }
```

And answer: who fills `RunOutcome` — the shim (converting `BestPoint.pose`) or does `Pose` cross as-is? (Reminder: it can't.)

Q5 — scope discipline. `POHSettings/newwithstrat`: exporting the strategy means making the enum's discriminants wire-contract today. Defer or commit? One-line decision.

Checkpoint: C++ smoke test (oracle-labeled is fine): bind a sphere cost, run budget=100, assert `outcome.value` within epsilon of the known min and calls `<= 100+overshoot`. That test is ALSO the first legitimate consumer of the previously-dead bridge link.

2.4 TODO Install a panic firewall [BLOCKER]

No `catch_unwind` exists anywhere in the crate; a panic inside any exported fn unwinds through C++ frames = UB/abort.

- Reachable panic #1: `.expect("array is fixed-size and non-empty")` in `Hyperbox::trisect` ([direct_{datastorage.rs}:86](#)) — called from run every iteration. Fix: `.map(|(i,_)| i).unwrap_or(0)` + comment why the fallback is provably dead (fixed `[u32;6]` can't be empty).
- Reachable panic #2: `cost.eval(&[physical])[0]` seed evaluation ([direct_{optimizer.rs}:86](#)) — unchecked index on a user-supplied vec.
- Then decide policy for everything exported: wrap each entry point in `catch_unwind -> status/Result` channel, or build the crate with `panic=abort`. Workspace lints are warn-only; nothing enforces this.

2.5 TODO Guard the C++ unbound-cost call [BLOCKER]

`CppCost::evaluate` calls `fn_(point)` without consulting its own `IsBound()` ([cost.h:42,45](#)). A default-constructed `CppCost` thrown across the boundary (`std::bad_function_call`) into Rust is UB territory once live. Wire the guard in (return NaN sentinel or abort loudly — pick and document), or forbid the default ctor from the FFI-facing surface.

2.6 TODO Namespace the bridge before signatures accrete

`#[cxx::bridge]` is bare ([lib.rs:19](#)); today's opaque types land in the global C++ namespace where `DirectOptimizer` collides conceptually with `::jta`'s own. The handshake doc already mandates it ([docs/rust-direct-rs-ffi-handshake.org:1224](#)): switch to `#[cxx::bridge(namespace = "direct_rs")]` NOW, while the surface is two types wide and the rename costs nothing.

3 TODO Close the numeric-parity gaps

parity

3.1 TODO Fix trisection axis selection (synthesis P1-2, highest leverage)

Rust splits the MINIMUM-depth axis (`rs: 81-88`); C++ splits the largest DENORMALIZED side ([direct_optimizer.cpp:247-250,329-331](#)). They coincide only for isotropic ranges — Tier-3 fails at the first anisotropic trisection. Either re-anchor the Rust split or prove JTML stage ranges are always isotropic (they aren't: trunk ranges are set per-axis in settings).

3.2 TODO Resolve box-id interleaving tie-break divergence

Rust assigns ALL center ids during trisect before any A/B ids (`rs: 168-190`): `order = oc1..ocN, A1,B1,A2,B2...` C++ inserts oc,A,B interleaved per POH box ([direct_optimizer.cpp:337,367-383](#)) and takes the oldest-inserted column minimum on ties. On cross-parent equal-cost ties within one size column the two sides select different boxes -> divergent POH sets next iteration. Structural finding, not yet demonstrated by a failing run — either re-order Rust's id assignment to interleave, or write the failing-case test proving it unreachable.

3.3 TODO Add `set_call_offset` (cumulative budget ladder)

The field exists but has no setter ([direct_optimizer.rs:12](#), hard-wired 0). Without it the trunk 20k -> branch 25k/30k -> leaf 35k cumulative ladder is inexpressible in the port. Mirror C++ `SetCallOffset` ([cpp:148-150](#)).

3.4 TODO Add the stop channel (`request_stop / AtomicBool`)

C++ has `Stop()/stop_requested_` checked between iterations ([cpp:115,144-146](#)). Rust has nothing — no flag, no way for C++ to cancel a running stage. Backed by `AtomicBool` per handshake doc open question.

3.5 TODO Decide the non-finite-eval contract

C++ counts non-finite evals and fires an extra iteration callback ([cpp:414-416](#)); Rust silently drops non-finite boxes with no counter (`rs: 191` guards insertion). If M2 progress callbacks ever ship this ordering matters; at minimum record the decision as out-of-parity-scope.

3.6 TODO Decide error channel: Result vs status-struct

Gated on whether anything in the Qt/CUDA toolchain builds with `-fno-exceptions` (grep found none, but never verified against toolchain defaults). This choice shapes EVERY exported fn signature — settle it before expanding the bridge.

3.7 TODO Build the Tier-3 parity fixture itself

No Rust-vs-C++ DIRECT replay test exists (all parity hits in test/ are QML-vs-widgets and CUDA-graph contracts). Need: recorded serial C++ cost tape + runner comparing call-for-call. Mind the serial-tape-vs- batch caveat from the review synthesis. Good news found by scouts: budget overshoot semantics MATCH on both sides (guard-at-loop-top, whole-batch eval — baseline.json:803 documents C++’s “+8 overshoot”; Rust mirrors it). That was open question 6 — RESOLVED, both sides identical.

4 TODO Wire the C++ side

wiring

4.1 TODO Land the A/B backend switch at the known insertion point

Everything funnels through RunDirectStage: the GPU cost is built unconditionally at [optimizer_manager.cpp:1315-1318](#) (`BuildGpuCostAdapter->DirectOptimizer opt(serial_cost, ...)`). A branch here swaps the fed cost. No OptimizerBackend enum exists yet; settings plumbing via `direct_options_` is already in place ([optimizer_manager.h:250-254](#)). Stage-loop call sites feeding it: `:1068, :1125, :1174`.

4.2 TODO Decide serial-vs-batch participation for the Rust backend

The batch path captures `serial_cost` by reference and may invoke it via banks ([optimizer_manager.cpp:1324-1351](#)). Threading fact scouts confirmed: RunDirectStage runs synchronously on the manager’s worker QThread, and the captured cost touches non-Send state (`gpu_principal_model_`, `stage_manager`). Today’s design (synchronous evaluate on the calling thread) avoids Send/Sync entirely — keep it that way, and decide explicitly whether the Rust branch forces the serial path or joins batch mode.

4.3 TODO Clean up the dead bridge link (or justify it)

`jtml_coordinator` PUBLIC-links `direct-rs_bridge` ([rust/CMakeLists.txt:19](#)) and `jtml_test_coordinator` links it ([test/CMakeLists.txt:622](#)) — but ZERO C++ TUs include `lib.rs.h` (repo-wide grep: docs only) and no test source uses it. Harmless today, misleading tomorrow: once the first real include lands, delete the test-target link unless a test genuinely drives the bridge.

4.4 TODO Reconcile the two parallel Corrosion mechanisms

`cmake/corrosion_cxx.cmake` defines `${stem}_cxxbridge` targets; `rust/CMakeLists.txt` uses Corrosion’s native `corrosion_add_cxxbridge` instead. One of them should go before someone wires the wrong one up.

4.5 TODO Guard or document the unconditional rust subdirectory

`add_subdirectory(rust)` is the last line of the top-level [CMakeLists.txt:132](#), no option guard — configure hard-requires a Rust toolchain via Corrosion’s REQUIRED discovery. Fine since pixi pins `rust >=1.97,<1.98` ([pixi.toml:47](#)), but a `JTML_ENABLE_RUST` option would let GPU-only environments configure.

4.6 TODO Pin the seam with headless unit tests

Zero tests reference `CppCost~/~build_cpp_cost` (grep of `test/`: 0 matches). Net-new pins, nothing breaks: bound/unbound evaluate behavior, build round-trip, and later the first `lib.rs.h`-consuming smoke test. Note the oracle-labeled tests that pin the production cost path (`bit_identitytest.cpp:234+`, `layered_correctnesstest.cpp:238+`) will be the A/B-switch acceptance surface — they are NOT in the `-L headless` fast set, so plan a deliberate oracle run when the switch lands.

4.7 TODO Fix or delete the broken cargo-build pixi task

`pixi.toml:157-158` uses `cargo build -m ...` — `-m` is not cargo's manifest flag (`-manifest-path` is). Task looks untested. Related (P2-13): there is still NO `cargo-test` task, so `pixi run test` never runs the Rust suite. Both belong in the gates.

5 TODO Idiomatic vs normie Rust

idiom

5.1 TODO Convert `#[allow(non_camel_case_types)]` to `#[expect(..., reason)]`

`direct_datastorage.rs:7` violates the workspace charter (`allow_attributes = "warn"`, `Cargo.toml:66-69`). Idiomatic form: `#[expect(non_camel_case_types, reason = "mirrors C++ Direction in data_structures")]`

5.2 TODO Delete the three dead artifacts

- Unused `use crate::ffi::CppCost;` at `direct_datastorage.rs:5` — sole occurrence in the file; a marker for glue that was never written.
- Dead private `trait Cost` (singular `f64` return) at `:37-39` — name collides with the real batch trait in `cost.rs`, implements nothing.
- Empty `src/pose.rs` — one blank line, not even declared in `lib.rs`.

5.3 TODO Replace the INFINITY sentinel with `Option<BestPoint>` internally

`current_best: (Pose, f64)` seeded with `f64::INFINITY` (`direct_optimizer.rs:39,57,124`) is the normie shape; the NaN-hostile property tests already work around it implicitly (`properties.rs:458-488`). Internally `Option<Best>`; at the boundary return a shared struct / bool-out-param instead of pushing INF across FFI. Pairs with the `BestPoint` newtype (below) — one refactor serving both.

5.4 TODO Introduce `BestPoint { pose, value }` newtype

Tuples leak through `new~/~new_with_strat~/~run~/~best` (`direct_optimizer.rs:8,36,82,128`). Improves internal readability AND becomes the natural `cxx` shared struct when those signatures cross.

5.5 TODO Drop fragile `PartialEq`, `PartialOrd` on `Pose`

Derived float ordering over six f64s ([direct_datastorage.rs:27](#)) is NaN-semantically dubious and `grep` finds no ordering user. Likely dead derives; remove rather than keep footguns.

5.6 TODO Harden the zip truncation hazard

`zip(unscored, evaluated_costs)` silently drops boxes if `Cost::eval` returns short ([direct_optimizer.rs:189](#)). One `debug_assert_eq!(len)` before it locks the batch-order contract the tests currently assert only for their own wrappers.

5.7 TODO Replace raw hull back-indexing

`hull[hull.len()-2] / hull[hull.len()-1]` ([direct_optimizer.rs:259-264](#)) — safe under the `len >= 2` guard but it is exactly the pattern the charter's `indexing_slicing` targets. Slice windows or `~get~+match` reads cleaner and silences the warn honestly.

5.8 TODO Purge the panic! in `plot_boxes`

[utils.rs:150](#) `unwrap_or_else(|_| panic!(..))` — plotting helper, but it is the crate's one production panic site; make it log-and-return.

5.9 TODO Fix the stale doc comment on the planned bridge signature

[cost.h:20](#) documents `fn evaluate(self: &CppCost, point: Point6D)` (by value) but reality is `point: &Point6D` ([lib.rs:31](#)). Harmless now, drift bait later.

5.10 TODO Naming: keep the status quo

Scouts confirmed `build_cpp_cost` is already `snakecase`, `evaluate` matches, and `cxx` auto-maps `snakecase` `<-> camelCase` for methods — no `cxx_name`/`~rust_name` needed for anything currently declared. Reserve renames for the `Direction`/`~POHSettings` enums if/when they cross as shared enums (then their `u32` discriminants become wire contract — choose them deliberately, today they are decorative).

6 Appendix — scout reports (full text)

Raw outputs preserved at:

- Bridge surface: `/ .pi/agent/sessions/--home-ajj-repo-uf-JTML--/subagent-artifacts/43efa504-8867-400e-a192-a963c05e7968_panoptes-explore_0_output.md`
- C++ call path: `.../8be963c1-9c5e-491b-9a0d-b31d28f62ae7_panoptes-explore_0_output.md`
- Contract cross-check: `.../bbc65122-ad4b-4a90-a961-8971e2389a8f_panoptes-explore_0_output.md`
- Idiom review: `.../e76b9a31-6787-4eb1-86fb-a4c4d59b6fe1_panoptes-explore_0_output.md`

Suggested attack order: panic firewall + namespace (cheap, do first) -> bridge expansion (`buildcppcost`, `Point6D` converter, exported surface + adapter) -> parity blockers (trisection axis, tie-break, `calloffset`, stop channel) -> A/B switch + tests -> idiom sweep.